

Reviewer Pack — Minimal Geometric Entropy of Tensor Products and DPLL Proof ...

Entry b15f261cf74a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 09:04:36 UTC

Minimal Geometric Entropy of Tensor Products and DPLL Proof Length

Entry ID: b15f261cf74a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 09:04:36 UTC

1. Conjecture

Field A (mathematical branch): Geometric Information Theory **Field B** (complexity object): DPLL Proof Complexity

Statement:

For every CNF φ with m clauses and n variables, the geometric entropy $H(\varphi)$ of its clause-tensor product is linearly correlated with its DPLL proof length $L_{\text{DPLL}}(\varphi)$, such that $H(\varphi) = \Theta(L_{\text{DPLL}}(\varphi))$.

Rationale (proposer's reasoning):

Geometric information theory provides tools to quantify randomness in geometric objects. The tensor product of clauses captures the interplay between different literals, which can influence the complexity of finding a satisfying assignment. A higher geometric entropy suggests more complex interactions, leading to longer DPLL proofs.

Taxonomy category: geometric_info_theory_to_dpll (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a99fe755e02d4e4f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the geometric entropy and DPLL proof length for all CNFs meets the condition: 'correlation_coefficient ≥ 0.7 AND p_value ≤ 0.05 '. It is falsified if either the correlation coefficient is less than 0.5 or the p-value exceeds 0.1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Minimal Geometric Entropy AND Tensor Products AND DPLL Proof Length - geometric entropy AND clause-tensor product AND DPLL proof length - Correlation between $H(\phi)$ AND $L_{DPLL}(\phi)$ in CNF ϕ

Top relevant hits considered: - [http://arxiv.org/abs/2312.09712v4] Geometric conservation in curved spacetime and entropy - [http://arxiv.org/abs/2403.13102v1] Geometric methods in quantum information and entanglement variational principle - [http://arxiv.org/abs/1005.2327v1] Black hole entropy in scalar-tensor and f(R) gravity: an overview - [http://arxiv.org/abs/2603.13936v2] Metric dimension and product entropy of group C^* -algebras - [http://arxiv.org/abs/2103.00503v6] Geometrical aspects of entropy production in stochastic thermodynamics based on Wasserstein distance - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_s^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2410.02421v2] Search for lepton number violating decays of $D_s^+ \rightarrow h^- h^0 e^+ e^+$ - [http://arxiv.org/abs/2604.05712v1] Precise measurement of the CKM angle γ with a novel approach

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def calculate_entropy(tensor_product):
    # Placeholder for actual entropy calculation using tensor products
    pass

def generate_cnf(num_clauses, num_vars):
    # Placeholder for CNF generation
    cnf = []
    for _ in range(num_clauses):
        clause = [random.choice([-1, 1]) * random.randint(1, num_vars) for _ in range(random.randint(1, num_vars))]
        cnf.append(clause)
    return cnf

def calculate_tensor_product(cnf):
    # Placeholder for tensor product calculation
    tensor_product = []
    for clause in cnf:
        new_clause = [random.choice([-1, 1]) * literal for literal in clause]
        tensor_product.extend(new_clause)
```

```

    return tensor_product

def run_dp11(cnf):
    # Placeholder for DPLL algorithm
    assignment = {}
    stack = [(cnf, assignment)]
    while stack:
        cnf, assignment = stack.pop()
        if not cnf:
            return 1
        unit_clauses = [c[0] for c in cnf if len(c) == 1]
        if unit_clauses:
            literal = unit_clauses[0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            stack.append([(l for l in c if l != literal and l != -literal) for c in cnf], new_assignment)
            continue
        literal, _ = random.choice(cnf)
        new_assignment1 = assignment.copy()
        new_assignment1[literal] = True
        stack.append([(c for c in cnf if literal not in c and -literal not in c), new_assignment1])
        new_assignment2 = assignment.copy()
        new_assignment2[literal] = False
        stack.append([(c for c in cnf if -literal not in c and literal not in c), new_assignment2])
    return 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    entropies = []
    lengths = []
    n_values = [5, 10, 15, 20, 30, 40]
    for n in n_values:
        for _ in range(5): # Ensure at least 30 instances per seed
            m = random.randint(n, n * 2)
            cnf = generate_cnf(m, n)
            tensor_product = calculate_tensor_product(cnf)
            entropy = calculate_entropy(tensor_product)
            dp11_length = run_dp11(cnf)
            entropies.append(entropy)
            lengths.append(dp11_length)
    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(entropies, lengths))
    p_value = 2 * (1 - math.erf(abs(correlation_coefficient) / math.sqrt(2 * len(entropies) - 3)))
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(entropies),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.7 and p_value <= 0.05,
        "counterexample": "" if correlation_coefficient >= 0.7 and p_value <= 0.05 else "correlation"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
results.append(result)

mean_x = sum(r["metric_value"] for r in results) / len(results)
std_x = math.sqrt(sum((r["metric_value"] - mean_x) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_x} std={std_x} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ed6e6df2.py", line 91, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ed6e6df2.py", line 73, in run_trial
    dpll_length = run_dpll(cnf)
                  ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ed6e6df2.py", line 53, in run_dpll
    literal, _ = random.choice(cnf)
                ^^^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the correlation coefficient and p-value could not be calculated to verify the conjecture. | next: Investigate the cause of the crash in the test code and run the test again to obtain valid results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14582
2	propose	ollama_remote	glm4:latest	0	0	21648
3	preregistration	ollama_remote	glm4:latest	0	0	19285
4	novelty	ollama_remote	glm4:latest	0	0	8487
5	novelty	ollama_remote	glm4:latest	0	0	9773
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21887
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16099
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9835
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12668
10	judge	ollama_remote	glm4:latest	0	0	14151

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148416 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/b15f261cf74a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/b15f261cf74a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/b15f261cf74a.tar.gz* (if generated)